**Tertiary Infotech Academy Pte Ltd**

UEN: 201200696W

# LEARNER GUIDE

For

## AI for Cyber Security

Course Code: C434

Conducted by

**Tertiary Infotech Academy Pte Ltd**

UEN: 201200696W

**Version 1.0**


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

## DOCUMENT VERSION CONTROL RECORD

| Version Number | Effective Date of Release | Summary of Included Changes | Author |
|---|---|---|---|
| 1.0 | 4 October 2026 | Initial release of AI for Cyber Security (C434): four topics, twelve detailed labs, evidence checks and current references. | Dr Alfred Ang |


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

## TABLE OF CONTENTS

**Course Overview** · purpose, outcomes and responsible-use boundary
**Topic 1: AI-Assisted Security Risk, Threat and Control-Gap Analysis** · LO1
- **Lab 1: Build an AI-Assisted Cyber Risk Register · LO1**
- **Lab 2: Detect Authentication Anomalies · LO1**
- **Lab 3: Explain a Phishing Risk Score · LO1**
- **Lab 4: Prioritise Vulnerabilities with Context · LO1**

**Topic 2: Designing AI-Powered Cybersecurity Programmes** · LO2
- **Lab 5: Select and Scope an AI Security Use Case · LO2**
- **Lab 6: Threat-Model an AI SOC Assistant · LO2**
- **Lab 7: Build the AI Security Programme Charter · LO2**

**Topic 3: Developing AI-Driven Security Architecture and Action Plans** · LO3
- **Lab 8: Design an AI-Enhanced SOC Architecture · LO3**
- **Lab 9: Build a Grounded Threat-Intelligence Assistant · LO3**
- **Lab 10: Design a Guarded SOAR Playbook · LO3**
- **Lab 11: Create a 90-Day AI Security Action Plan · LO3**

**Topic 4: Monitoring, Evaluating and Governing AI Security Solutions** · LO4
- **Lab 12: Monitor Model Effectiveness and Produce Advice · LO4**

**References** · current primary and authoritative sources


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

## Course Overview

This two-day, 15-hour course develops defensible uses of AI across cyber risk analysis, programme design, security architecture, monitoring and governance.

## Learning Outcomes

- LO1: Identify security risks, threats, and assess gaps in controls impacting business processes.
- LO2: Translate security objectives into specific programmes considering security programme design techniques.
- LO3: Develop a detailed action plan for security programmes in line with information security architectures.
- LO4: Deliver advice for adoption of security architectures, monitoring their effectiveness against standards.

## Responsible Use Boundary

AI output is a hypothesis, not a verdict. Preserve raw evidence, minimise sensitive data, retain provenance, state confidence and limitations, and require human approval for high-impact actions.

## Topic 1: AI-Assisted Security Risk, Threat and Control-Gap Analysis

Alignment: LO1. Telemetry contracts · detection models · business impact · analyst evidence

### Security telemetry is a governed evidence pipeline

Runtime mechanism: Collect -> Normalize -> Enrich -> Score -> Retain

| Contract field / rule | Operational meaning |
|---|---|
| event_time: UTC timestamp | Schema-first features survive tool changes |
| principal.id: stable identity | Late events need event-time windows |
| asset.criticality: 1-5 | Asset context changes the same alert's risk |
| action/outcome: attempted result | Raw evidence remains immutable |

#### Failure, control and evidence

| Failure mode | Control / evidence |
|---|---|
| Clock skew creates false sequences | Parse success >= 99% |
| Missing identity joins hide lateral movement | Field null rates tracked |
| PII copied into prompts | Lineage links score to raw event |
| Silent parser failures | Retention matches policy |

Practice bridge: Lab 1 applies this mechanism; the full procedure follows in this guide and the individual lab folder.


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

## Risk scoring joins threat likelihood to business impact

Runtime mechanism: Threat -> Exposure -> Control gap -> Impact -> Decision

| Contract field / rule | Operational meaning |
|---|---|
| risk = likelihood x impact | CVSS measures severity, not organisational priority |
| likelihood = exploitability x exposure | EPSS estimates exploitation probability |
| residual = inherent x (1-control_effectiveness) | Criticality weights business consequence |
| owner + due_date + evidence | Confidence must travel with the score |

### Failure, control and evidence

| Failure mode | Control / evidence |
|---|---|
| Single-number theatre | Top risks trace to assets |
| Unbounded AI-generated severity | Sensitivity analysis recorded |
| No risk owner | Human approves priority band |
| No exception expiry | Exceptions expire automatically |

Practice bridge: Lab 1 applies this mechanism; the full procedure follows in this guide and the individual lab folder.

## Anomaly detection models normal behaviour, not malicious intent

Runtime mechanism: Baseline -> Feature vector -> Distance score -> Threshold -> Analyst review

| Contract field / rule | Operational meaning |
|---|---|
| window=15m | Isolation Forest isolates sparse points |
| features=[failures,new_device,bytes_out] | Z-score assumes a stable distribution |
| score = model.decision_function(x) | Seasonality needs peer/time baselines |
| alert if score < threshold | Threshold controls workload and misses |

### Failure, control and evidence

| Failure mode | Control / evidence |
|---|---|
| Novel but benign deployment | Backtest on labelled window |
| Poisoned baseline | Report precision and recall |
| Feature drift | Analyst disposition captured |
| Rare user treated as hostile | Threshold reviewed monthly |

Practice bridge: Lab 2 applies this mechanism; the full procedure follows in this guide and the individual lab folder.

## Phishing detection combines header, URL and language evidence

Runtime mechanism: Parse mail -> Extract features -> Classify -> Explain -> Quarantine/review

| Contract field / rule | Operational meaning |
|---|---|
| spf/dkim/dmarc result | Header authentication is stronger than tone alone |
| reply_to mismatch | URL structure adds campaign signal |
| url_domain_age_days | Text models require multilingual evaluation |
| language urgency score | Explanations should cite observed fields |

### Failure, control and evidence

| Failure mode | Control / evidence |
|---|---|
| Attacker prompt text manipulates an LLM | Evidence cites exact header/URL |
| Legitimate urgent mail | No auto-delete at low confidence |
| Image-only lure | False positives sampled weekly |


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

| Failure mode | Control / evidence |
|---|---|
| Domain reputation delay | Protected sandbox for links |

Practice bridge: Lab 3 applies this mechanism; the full procedure follows in this guide and the individual lab folder.

## Malware triage separates static signals from behavioural proof

Runtime mechanism: Hash/type -> Static features -> Sandbox -> Classifier -> Disposition

| Contract field / rule | Operational meaning |
|---|---|
| sha256 + file_type | Static ML scales pre-screening |
| imports/entropy/sections | Behaviour reveals packed samples |
| process/network/file behaviours | Family labels are noisy |
| family confidence + evidence | Unknown stays unknown |

### Failure, control and evidence

| Failure mode | Control / evidence |
|---|---|
| Adversarial packing | No live malware in training |
| Sandbox evasion | Hash and provenance recorded |
| Training labels from one vendor | Confidence threshold documented |
| Executing samples outside isolation | Analyst confirms containment |

Practice bridge: Lab 4 applies this mechanism; the full procedure follows in this guide and the individual lab folder.

## Vulnerability prioritisation is a multi-signal ranking problem

Runtime mechanism: Inventory -> Severity -> Exploit likelihood -> Exposure -> Business context

| Contract field / rule | Operational meaning |
|---|---|
| asset_id + owner | Known exploitation outranks theoretical severity |
| CVSS base/vector | Exposure time affects urgency |
| EPSS probability | Compensating controls lower residual risk |
| internet_exposed + compensating_control | AI summarises evidence; policy sets SLA |

### Failure, control and evidence

| Failure mode | Control / evidence |
|---|---|
| Hallucinated CVE details | Source URL and retrieval date |
| Stale inventory | Reachability evidence |
| Dependency not reachable | Priority rationale stored |
| SLA ignores business owner | SLA breach dashboard |

Practice bridge: Lab 5 applies this mechanism; the full procedure follows in this guide and the individual lab folder.

## UEBA needs peer groups, identity joins and privacy controls

Runtime mechanism: Identity graph -> Peer baseline -> Session features -> Risk aggregation -> Investigation

| Contract field / rule | Operational meaning |
|---|---|
| principal.id + role | Peer grouping reduces role-driven noise |
| device.id + managed | Risk accumulates across weak signals |
| geo_velocity_kmh | Identity resolution precedes analytics |
| privilege_change + data_access | Privacy purpose limits remain |


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

### Failure, control and evidence

| Failure mode | Control / evidence |
|---|---|
| Shared accounts | Purpose documented |
| Role change not reflected | Access restricted |
| Bias against remote workers | Appeal/correction route |
| Excessive employee surveillance | Alert evidence reproducible |

Practice bridge: Lab 2 applies this mechanism; the full procedure follows in this guide and the individual lab folder.

## Threat-intelligence extraction produces structured, confidence-scored objects

Runtime mechanism: Source -> Extract -> Normalise -> Correlate -> Operationalise

| Contract field / rule | Operational meaning |
|---|---|
| indicator.type/value | NLP accelerates entity extraction |
| valid_from/valid_until | STIX-like structure enables joins |
| confidence 0-100 | Confidence depends on source and corroboration |
| source + relationship | Expiry prevents permanent blocks |

### Failure, control and evidence

| Failure mode | Control / evidence |
|---|---|
| Defanged indicators parsed wrongly | Schema validation |
| Duplicate aliases | Provenance retained |
| Untrusted report instructions | Human approves blocking |
| IOC without context | Expired indicators removed |

Practice bridge: Lab 6 applies this mechanism; the full procedure follows in this guide and the individual lab folder.

## Lab 1: Build an AI-Assisted Cyber Risk Register

Mapped outcome: LO1 · Tools: Python 3 · CSV · JSON · Cybersecurity Threat Simulation Lab

Join asset criticality, threat likelihood, control effectiveness and evidence into a transparent residual-risk ranking.

### Controlled simulator or Kali-style practice

Use Cybersecurity Threat Simulation Lab (https://alfredang.github.io/cybersecuritysimulator/) in its Threat Dashboard module. Observe the simulated threat categories and translate three observations into risk-register evidence references and control-gap notes. Follow tool-exercise.md in the individual lab folder. The files in mock-data/ provide a fully offline assessment-equivalent path.

Safety boundary: use the browser simulation or a trainer-owned isolated Kali target only. Do not scan or test unrelated live systems.


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

### Detailed step-by-step procedure

1. **Review the asset, threat and control columns in data/input.csv.**

   Evidence — record the input, configuration or threshold, generated output and decision rationale.
   Check — All rows have owner and evidence_ref

2. **Run the starter script and inspect its deliberately incomplete residual-risk calculation.**

   Evidence — record the input, configuration or threshold, generated output and decision rationale.
   Check — Residual scores are 0-25

3. **Implement likelihood x impact and control-effectiveness reduction without overwriting raw evidence.**

   Evidence — record the input, configuration or threshold, generated output and decision rationale.
   Check — Ranking changes when asset criticality changes

4. **Rank the risks, assign owners and record a confidence note for each row.**

   Evidence — record the input, configuration or threshold, generated output and decision rationale.
   Check — Output JSON records the scoring formula

5. **Compare the ranking with the business-process impact column and document one changed priority.**

   Evidence — record the input, configuration or threshold, generated output and decision rationale.
   Check — Output JSON records the scoring formula

6. **Run the smoke test and verify that every output row retains asset_id and evidence_ref.**

   Evidence — record the input, configuration or threshold, generated output and decision rationale.
   Check — Output JSON records the scoring formula

### Run and verify

```
cd labs/lab-01-build-an-ai-assisted-cyber-risk-register
python3 solution/main.py
python3 -m pytest -q tests
```

### Acceptance criteria

- All rows have owner and evidence_ref
- Residual scores are 0-25


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Ranking changes when asset criticality changes
- Output JSON records the scoring formula

### Troubleshooting

If the script cannot find data, run it from the lab folder. If scores are unexpected, inspect type conversion, null handling, baseline/evaluation split and threshold direction. Do not lower a threshold merely to make a test pass; explain the operational effect.

## Lab 2: Detect Authentication Anomalies

Mapped outcome: LO1 · Tools: Python 3 · pandas · scikit-learn · FauxBank Cybersecurity Training Simulation

Create a peer-aware anomaly score from failed logins, new-device state and impossible-travel indicators.

### Controlled simulator or Kali-style practice

Use FauxBank Cybersecurity Training Simulation (https://pentest-fauxbank.vercel.app/) in its Fictional sign-in and account activity module. Compare weak-credential and account-behaviour observations with the local authentication baseline; treat the site as a reversible education sandbox only. Follow tool-exercise.md in the individual lab folder. The files in mock-data/ provide a fully offline assessment-equivalent path.

Safety boundary: use the browser simulation or a trainer-owned isolated Kali target only. Do not scan or test unrelated live systems.

### Detailed step-by-step procedure

7. **Inspect the authentication schema and identify event-time, identity and device fields.**

   Evidence — record the input, configuration or threshold, generated output and decision rationale.
   Check — Model trained only on baseline rows
8. **Create 15-minute features per user.**

   Evidence — record the input, configuration or threshold, generated output and decision rationale.
   Check — Output includes anomaly_score and reason fields
9. **Fit IsolationForest only on the baseline rows.**

   Evidence — record the input, configuration or threshold, generated output and decision rationale.
   Check — Queue has at most three alerts


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

10. **Score the evaluation rows and preserve reason features.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Threshold and model random_state are recorded

11. **Tune the percentile threshold to a maximum review queue of three alerts.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Threshold and model random_state are recorded

12. **Record false-positive risks and run the tests.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Threshold and model random_state are recorded

### Run and verify

```
cd labs/lab-02-detect-authentication-anomalies
python3 solution/main.py
python3 -m pytest -q tests
```

### Acceptance criteria

- Model trained only on baseline rows
- Output includes anomaly_score and reason fields
- Queue has at most three alerts
- Threshold and model random_state are recorded

### Troubleshooting

If the script cannot find data, run it from the lab folder. If scores are unexpected, inspect type conversion, null handling, baseline/evaluation split and threshold direction. Do not lower a threshold merely to make a test pass; explain the operational effect.


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

## Lab 3: Explain a Phishing Risk Score

Mapped outcome: LO1 · Tools: Python 3 · pandas · Cybersecurity Threat Simulation Lab

Combine email authentication, reply-to mismatch, URL and language features into an explainable phishing triage score.

### Controlled simulator or Kali-style practice

Use Cybersecurity Threat Simulation Lab (https://alfredang.github.io/cybersecuritysimulator/) in its Phishing simulator and URL analyser module. Classify the fictional messages and URL indicators, then compare the simulator rationale with locally generated reason codes. Follow tool-exercise.md in the individual lab folder. The files in mock-data/ provide a fully offline assessment-equivalent path.

Safety boundary: use the browser simulation or a trainer-owned isolated Kali target only. Do not scan or test unrelated live systems.

### Detailed step-by-step procedure

13. **Review the safely simulated email feature table.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — No raw email body leaves the lab

14. **Assign weights to SPF/DKIM failure, reply-to mismatch, young domains and urgency.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Every score has at least one reason code

15. **Calculate a 0-100 risk score.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Low-confidence mail is review-only

16. **Generate reason codes from observed fields only.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Thresholds are documented

17. **Apply review/quarantine thresholds with human approval for destructive action.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Thresholds are documented


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

18. **Test a legitimate urgent message and document the false-positive control.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Thresholds are documented

#### Run and verify

```
cd labs/lab-03-explain-a-phishing-risk-score
python3 solution/main.py
python3 -m pytest -q tests
```

#### Acceptance criteria

- No raw email body leaves the lab
- Every score has at least one reason code
- Low-confidence mail is review-only
- Thresholds are documented

#### Troubleshooting

If the script cannot find data, run it from the lab folder. If scores are unexpected, inspect type conversion, null handling, baseline/evaluation split and threshold direction. Do not lower a threshold merely to make a test pass; explain the operational effect.

### Lab 4: Prioritise Vulnerabilities with Context

Mapped outcome: LO1 · Tools: Python 3 · pandas · Hacklab: Ethical Hacking Lab Simulator

Rank vulnerabilities using severity, exploit likelihood, exposure, asset criticality and compensating controls.

#### Controlled simulator or Kali-style practice

Use Hacklab: Ethical Hacking Lab Simulator (https://alfredang.github.io/ethnicalhacking/) in its Vulnerability assessment scenario module. Use the simulated terminal to collect vulnerability evidence, then combine it with criticality and compensating-control data before setting remediation priority. Follow tool-exercise.md in the individual lab folder. The files in mock-data/ provide a fully offline assessment-equivalent path.

Safety boundary: use the browser simulation or a trainer-owned isolated Kali target only. Do not scan or test unrelated live systems.

#### Detailed step-by-step procedure

19. **Inspect the synthetic CVE inventory and verify source/retrieval dates.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — CVSS and EPSS remain separate fields


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

20. **Calculate a transparent contextual priority score.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Internet exposure changes priority

21. **Compare the result with a CVSS-only sort.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Every row keeps source and retrieval_date

22. **Assign remediation SLAs by policy bands.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — SLA is policy-derived

23. **Record one case where a lower-CVSS item moves ahead.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — SLA is policy-derived

24. **Run tests for score bounds and required provenance.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — SLA is policy-derived

### Run and verify

```
cd labs/lab-04-prioritise-vulnerabilities-with-context
python3 solution/main.py
python3 -m pytest -q tests
```

### Acceptance criteria

- CVSS and EPSS remain separate fields
- Internet exposure changes priority
- Every row keeps source and retrieval_date
- SLA is policy-derived

### Troubleshooting

If the script cannot find data, run it from the lab folder. If scores are unexpected, inspect type conversion, null handling, baseline/evaluation split and threshold direction. Do not lower a threshold merely to make a test pass; explain the operational effect.


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

# Topic 2: Designing AI-Powered Cybersecurity Programmes

Alignment: LO2. Objectives · use-case selection · governance · human oversight · operating model

## Programme design starts with an outcome, not an AI product

Runtime mechanism: Security objective -> Decision -> Evidence -> Control -> Metric

| Contract field / rule | Operational meaning |
|---|---|
| problem statement | Use cases compete on value, feasibility and risk |
| decision owner | Baseline defines improvement |
| baseline + target | Decision rights prevent shadow automation |
| automation ceiling | Kill criteria stop sunk-cost escalation |

### Failure, control and evidence

| Failure mode | Control / evidence |
|---|---|
| Technology-first purchase | Named owner |
| No measurable baseline | Baseline frozen |
| Unowned alerts | Success/stop thresholds |
| Pilot becomes production silently | Review date |

Practice bridge: Lab 7 applies this mechanism; the full procedure follows in this guide and the individual lab folder.

## Control gaps translate into programme epics and guardrails

Runtime mechanism: Current control -> Gap evidence -> Security objective -> Programme epic -> Acceptance

| Contract field / rule | Operational meaning |
|---|---|
| control_id | A gap is an evidence-backed delta |
| current_state/evidence | Objectives state the desired security outcome |
| target_state | Epics group related controls |
| owner/milestone/dependency | Acceptance criteria remain testable |

### Failure, control and evidence

| Failure mode | Control / evidence |
|---|---|
| Vague 'improve security' objective | Gap references audit/log |
| AI invents missing evidence | Target has date |
| Control owner excluded | Owner accepts scope |
| No dependency map | Test plan exists |

Practice bridge: Lab 7 applies this mechanism; the full procedure follows in this guide and the individual lab folder.

## Data governance is part of the security control plane

Runtime mechanism: Classify -> Minimise -> Authorise -> Protect -> Delete

| Contract field / rule | Operational meaning |
|---|---|
| data class | Sensitive telemetry can identify people |
| allowed purpose | Prompts are data transfers |
| retention | Training use must be explicit |
| region | Deletion must cover caches and indexes |
| model/provider use | Deletion must cover caches and indexes |


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

### Failure, control and evidence

| Failure mode | Control / evidence |
|---|---|
| Secrets in prompts | DLP test |
| Unbounded retention | Access log |
| Cross-border transfer | Deletion evidence |
| Vendor trains on customer data | Vendor contract control |

Practice bridge: Lab 8 applies this mechanism; the full procedure follows in this guide and the individual lab folder.

## Automation levels define where humans must decide

Runtime mechanism: Recommend -> Draft -> Approve -> Execute -> Reverse

| Contract field / rule | Operational meaning |
|---|---|
| confidence threshold | Low-impact enrichment can be automatic |
| action allow-list | Containment needs bounded targets |
| approval role | High-impact identity actions need approval |
| rollback command | Reversibility lowers operational risk |

### Failure, control and evidence

| Failure mode | Control / evidence |
|---|---|
| Model blocks a critical account | Dry-run default |
| Approval fatigue | Two-person approval for destructive action |
| No rollback | Full audit trail |
| Action scope expands via tool permissions | Quarterly access review |

Practice bridge: Lab 9 applies this mechanism; the full procedure follows in this guide and the individual lab folder.

## Threat modelling covers both AI use and the AI system

Runtime mechanism: Assets -> Trust boundaries -> Abuse cases -> Controls -> Tests

| Contract field / rule | Operational meaning |
|---|---|
| model/data/tool identities | AI can strengthen defence and create attack paths |
| prompt/retrieval/tool boundaries | Tool permissions are a primary boundary |
| STRIDE + ATLAS techniques | Indirect prompt injection travels through content |
| security test case | Model supply chain needs provenance |

### Failure, control and evidence

| Failure mode | Control / evidence |
|---|---|
| Prompt injection | Abuse case mapped |
| Training-data poisoning | Control owner |
| Model theft | Test evidence |
| Unsafe tool invocation | Residual risk accepted |

Practice bridge: Lab 8 applies this mechanism; the full procedure follows in this guide and the individual lab folder.

## Build-versus-buy decisions depend on control and evidence

Runtime mechanism: Requirements -> Market scan -> Evidence request -> Risk decision -> Contract

| Contract field / rule | Operational meaning |
|---|---|
| data use/retention | Embedded AI reduces integration effort |
| model update notice | Custom models increase ownership burden |
| tenant isolation | Benchmarks must match local data |


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

| Contract field / rule | Operational meaning |
|---|---|
| incident notification | Exit plans prevent lock-in |

### Failure, control and evidence

| Failure mode | Control / evidence |
|---|---|
| Benchmark without dataset details | Proof-of-value dataset |
| No audit logs | Security questionnaire |
| Unannounced model changes | Contractual controls |
| Provider dependency | Export/delete test |

Practice bridge: Lab 8 applies this mechanism; the full procedure follows in this guide and the individual lab folder.

## Secure adoption applies least privilege to models, tools and people

Runtime mechanism: Authenticate -> Authorise -> Constrain tools -> Monitor -> Review

| Contract field / rule | Operational meaning |
|---|---|
| service identity | The model is not an identity |
| scoped retrieval index | Every tool call uses a governed principal |
| tool allow-list | Retrieval permissions filter before generation |
| per-action audit event | Logs capture inputs, decisions and actions |

### Failure, control and evidence

| Failure mode | Control / evidence |
|---|---|
| Shared API key | RBAC test |
| Cross-tenant retrieval | Tenant-isolation test |
| Over-privileged plugin | Secret scan |
| Sensitive log content exposed | Access recertification |

Practice bridge: Lab 9 applies this mechanism; the full procedure follows in this guide and the individual lab folder.

## The operating model links people, process and technology

Runtime mechanism: Govern -> Product owner -> SOC operations -> Model risk -> Audit

| Contract field / rule | Operational meaning |
|---|---|
| RACI | Product ownership outlives the pilot |
| change window | SOC owns operational disposition |
| incident route | Model risk owns validation |
| model card + runbook | Audit needs evidence independent of the vendor |

### Failure, control and evidence

| Failure mode | Control / evidence |
|---|---|
| Nobody owns false positives | RACI approved |
| Change bypasses CAB | On-call route tested |
| Runbook lacks fallback | Fallback exercised |
| Audit depends on screenshots | Evidence pack reproducible |

Practice bridge: Lab 7 applies this mechanism; the full procedure follows in this guide and the individual lab folder.


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

## Lab 5: Select and Scope an AI Security Use Case

Mapped outcome: LO2 · Tools: Python 3 · CSV · Markdown · Cybersecurity Threat Simulation Lab

Score candidate use cases on value, data readiness, integration effort, operating risk and human-oversight requirements.

### Controlled simulator or Kali-style practice

Use Cybersecurity Threat Simulation Lab (https://alfredang.github.io/cybersecuritysimulator/) in its Threat Dashboard and scenario modules module. Identify candidate AI-support opportunities from the simulated scenarios, then score value, readiness, integration effort and operating risk. Follow tool-exercise.md in the individual lab folder. The files in mock-data/ provide a fully offline assessment-equivalent path.

Safety boundary: use the browser simulation or a trainer-owned isolated Kali target only. Do not scan or test unrelated live systems.

### Detailed step-by-step procedure

25. **Define the security outcome and decision owner.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Selected use case has measurable baseline

26. **Score each use case against the provided rubric.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Decision owner named

27. **Apply a risk penalty for high-impact autonomous action.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Automation ceiling explicit

28. **Select one pilot and state the baseline and target.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Stop criteria included

29. **Set automation ceiling, stop criteria and review date.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Stop criteria included


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

30. **Peer-review the charter against LO2.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Stop criteria included

### Run and verify

```
cd labs/lab-05-select-and-scope-an-ai-security-use-case
python3 solution/main.py
python3 -m pytest -q tests
```

### Acceptance criteria

- Selected use case has measurable baseline
- Decision owner named
- Automation ceiling explicit
- Stop criteria included

### Troubleshooting

If the script cannot find data, run it from the lab folder. If scores are unexpected, inspect type conversion, null handling, baseline/evaluation split and threshold direction. Do not lower a threshold merely to make a test pass; explain the operational effect.

## Lab 6: Threat-Model an AI SOC Assistant

Mapped outcome: LO2 · Tools: Python 3 · JSON · MITRE ATLAS/OWASP references · Hacklab: Ethical Hacking Lab Simulator

Map assets, trust boundaries and AI-specific abuse cases, then turn them into programme guardrails.

### Controlled simulator or Kali-style practice

Use Hacklab: Ethical Hacking Lab Simulator (https://alfredang.github.io/ethnicalhacking/) in its Malware analysis scenario module. Collect static-analysis observations from the fictional sample and model how untrusted evidence could manipulate an AI SOC assistant. Follow tool-exercise.md in the individual lab folder. The files in mock-data/ provide a fully offline assessment-equivalent path.

Safety boundary: use the browser simulation or a trainer-owned isolated Kali target only. Do not scan or test unrelated live systems.

### Detailed step-by-step procedure

31. **Draw the user-model-retrieval-tool-target trust boundaries.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — At least four trust boundaries


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

32. **Identify direct and indirect prompt-injection paths.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Every abuse case has a control owner

33. **Map data leakage, unsafe tool invocation and poisoned feedback risks.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Tool actions are allow-listed

34. **Assign preventive, detective and recovery controls.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Residual risk is stated

35. **Name control owners and acceptance tests.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Residual risk is stated

36. **Validate the JSON schema and peer-review residual risk.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Residual risk is stated

### Run and verify

```
cd labs/lab-06-threat-model-an-ai-soc-assistant
python3 solution/main.py
python3 -m pytest -q tests
```

### Acceptance criteria

- At least four trust boundaries
- Every abuse case has a control owner
- Tool actions are allow-listed
- Residual risk is stated

### Troubleshooting

If the script cannot find data, run it from the lab folder. If scores are unexpected, inspect type conversion, null handling, baseline/evaluation split and threshold direction. Do not lower a threshold merely to make a test pass; explain the operational effect.


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

## Lab 7: Build the AI Security Programme Charter

Mapped outcome: LO2 · Tools: Markdown · CSV · policy template · FauxBank Cybersecurity Training Simulation

Translate assessed control gaps into governed programme epics, metrics, owners and decision gates.

### Controlled simulator or Kali-style practice

Use FauxBank Cybersecurity Training Simulation (https://pentest-fauxbank.vercel.app/) in its Guided findings and report generator module. Turn the fictional application findings into programme epics, accountable ownership, metrics and decision gates. Follow tool-exercise.md in the individual lab folder. The files in mock-data/ provide a fully offline assessment-equivalent path.

Safety boundary: use the browser simulation or a trainer-owned isolated Kali target only. Do not scan or test unrelated live systems.

### Detailed step-by-step procedure

37. **Select the highest-priority control gaps from the lab data.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.

    Check — Every epic traces to a gap

38. **Write outcome-based objectives and measurable targets.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.

    Check — Metric has baseline and target

39. **Group work into data, model, integration, people and assurance epics.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.

    Check — RACI has one accountable owner

40. **Create RACI assignments and an escalation route.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.

    Check — Retirement/exit gate exists

41. **Define pilot, production and retirement gates.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.

    Check — Retirement/exit gate exists


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

42. **Run the charter checklist and resolve missing owners.**

Evidence — record the input, configuration or threshold, generated output and decision rationale.
Check — Retirement/exit gate exists

### Run and verify

```
cd labs/lab-07-build-the-ai-security-programme-charter
python3 solution/main.py
python3 -m pytest -q tests
```

### Acceptance criteria

- Every epic traces to a gap
- Metric has baseline and target
- RACI has one accountable owner
- Retirement/exit gate exists

### Troubleshooting

If the script cannot find data, run it from the lab folder. If scores are unexpected, inspect type conversion, null handling, baseline/evaluation split and threshold direction. Do not lower a threshold merely to make a test pass; explain the operational effect.

## Topic 3: Developing AI-Driven Security Architecture and Action Plans

Alignment: LO3. SOC architecture · data pipeline · guarded automation · delivery roadmap

### An AI-enhanced SOC separates evidence, inference and action

Runtime mechanism: Telemetry -> Security lake/SIEM -> Analytics -> Case -> Guarded response

| Contract field / rule | Operational meaning |
|---|---|
| immutable event store | Evidence remains queryable without the model |
| feature/model service | Inference is versioned |
| case evidence bundle | Cases preserve human disposition |
| SOAR approval gate | Response uses scoped service accounts |

### Failure, control and evidence

| Failure mode | Control / evidence |
|---|---|
| LLM directly queries production | Replay creates same result |
| No model version | Model/version logged |
| Alert overwrites raw evidence | Case links raw events |
| SOAR action lacks approval | Rollback tested |

Practice bridge: Lab 9 applies this mechanism; the full procedure follows in this guide and the individual lab folder.


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

## Feature pipelines prevent training-serving skew

Runtime mechanism: Raw events -> Transform -> Feature store -> Train -> Serve

| Contract field / rule | Operational meaning |
|---|---|
| feature name/type | One definition serves training and inference |
| window and timezone | Point-in-time joins prevent leakage |
| default/null policy | Feature freshness is monitored |
| owner/version | Backfills are versioned |

### Failure, control and evidence

| Failure mode | Control / evidence |
|---|---|
| Future data leaks into training | Offline/online parity test |
| Different parser in production | Feature freshness SLO |
| Null default changes | Schema contract test |
| Stale enrichment | Backfill audit |

Practice bridge: Lab 10 applies this mechanism; the full procedure follows in this guide and the individual lab folder.

## SIEM plus ML uses rules and models as complementary detectors

Runtime mechanism: Rule match -> Model score -> Correlation -> Case severity -> Disposition

| Contract field / rule | Operational meaning |
|---|---|
| rule_id + version | Rules encode known policy |
| model_id + threshold | Models surface weak novel patterns |
| entity graph | Correlation reduces isolated noise |
| reason codes | Disposition feeds both tuning loops |

### Failure, control and evidence

| Failure mode | Control / evidence |
|---|---|
| Model replaces high-confidence rule | Detection-as-code test |
| Duplicate alerts | Duplicate suppression |
| Correlation window too wide | Latency SLO |
| Feedback label leakage | Tuning change record |

Practice bridge: Lab 10 applies this mechanism; the full procedure follows in this guide and the individual lab folder.

## RAG grounds an analyst assistant in approved evidence

Runtime mechanism: Curate -> Chunk -> Retrieve -> Generate -> Cite

| Contract field / rule | Operational meaning |
|---|---|
| document_id/version | Retrieval narrows evidence |
| chunk + ACL | ACL filtering precedes generation |
| top_k + similarity | Citations enable verification |
| answer + citations | Abstention is a valid output |

### Failure, control and evidence

| Failure mode | Control / evidence |
|---|---|
| Indirect prompt injection | Citation precision |
| Stale runbook | Forbidden-document test |
| Cross-case retrieval | Freshness check |
| Confident uncited answer | Abstention threshold |


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Practice bridge: Lab 10 applies this mechanism; the full procedure follows in this guide and the individual lab folder.

## Identity boundaries control every AI-to-tool transition

Runtime mechanism: User -> Application -> Model -> Tool broker -> Target system

| Contract field / rule | Operational meaning |
|---|---|
| user/session claims | The tool broker mediates calls |
| workload identity | Delegation cannot exceed the user |
| delegated scope | Secrets stay outside prompts |
| policy decision + audit | Break-glass is separate |

### Failure, control and evidence

| Failure mode | Control / evidence |
|---|---|
| Model holds admin token | Token scope test |
| Confused deputy | Denied-action test |
| Long-lived secret | Secret rotation |
| Unlogged break-glass | Break-glass review |

Practice bridge: Lab 9 applies this mechanism; the full procedure follows in this guide and the individual lab folder.

## Guarded SOAR uses policy gates and reversible actions

Runtime mechanism: Enrich -> Recommend -> Simulate -> Approve -> Execute/rollback

| Contract field / rule | Operational meaning |
|---|---|
| action type | Simulation checks blast radius |
| target scope | Preconditions prevent stale actions |
| precondition | Approval follows impact tier |
| approval | Rollback is part of the playbook |
| rollback | Rollback is part of the playbook |

### Failure, control and evidence

| Failure mode | Control / evidence |
|---|---|
| Race condition | Dry-run output |
| Wrong entity | Approval record |
| Partial execution | Post-action verification |
| Rollback dependency unavailable | Rollback exercise |

Practice bridge: Lab 11 applies this mechanism; the full procedure follows in this guide and the individual lab folder.

## An action plan converts architecture into sequenced delivery

Runtime mechanism: Current state -> Target state -> Work packages -> Dependencies -> Milestones

| Contract field / rule | Operational meaning |
|---|---|
| epic/control | Architecture decisions constrain work |
| owner | Critical path exposes sequencing |
| start/end | Pilot and production have different gates |
| dependency | Regulatory changes trigger backlog review |
| acceptance evidence | Regulatory changes trigger backlog review |


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

### Failure, control and evidence

| Failure mode | Control / evidence |
|---|---|
| Roadmap has dates but no dependencies | Dependency review |
| Control evidence deferred | Milestone evidence |
| No decommission task | Risk burn-down |
| Owner changes silently | Change log |

Practice bridge: Lab 11 applies this mechanism; the full procedure follows in this guide and the individual lab folder.

## Resilience assumes models and providers can fail

Runtime mechanism: Detect failure -> Degrade safely -> Fallback -> Recover -> Learn

| Contract field / rule | Operational meaning |
|---|---|
| availability/latency SLO | Security fundamentals remain available |
| manual/rule fallback | Fail-closed is not always safe |
| backup configuration | Fallback needs equivalent audit |
| recovery objective | Recovery includes model and index versions |

### Failure, control and evidence

| Failure mode | Control / evidence |
|---|---|
| Provider outage | Game-day exercise |
| Corrupt vector index | Manual queue works |
| Rate-limit spike | Restore checksum |
| Bad model rollout | Post-incident action |

Practice bridge: Lab 11 applies this mechanism; the full procedure follows in this guide and the individual lab folder.

## Lab 8: Design an AI-Enhanced SOC Architecture

Mapped outcome: LO3 · Tools: JSON · architecture template · Cryptography Toolkit

Design a layered architecture that separates evidence, inference, case management and guarded action.

### Controlled simulator or Kali-style practice

Use Cryptography Toolkit (https://alfredang.github.io/cryptography-toolkit/) in its ECDSA sign and verify module. Sign a fictional architecture decision record and verify it after a controlled text change to demonstrate integrity evidence, not identity assurance by itself. Follow tool-exercise.md in the individual lab folder. The files in mock-data/ provide a fully offline assessment-equivalent path.

Safety boundary: use the browser simulation or a trainer-owned isolated Kali target only. Do not scan or test unrelated live systems.

### Detailed step-by-step procedure

43. **Inventory telemetry sources and owners.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Raw evidence is immutable


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

44. **Define immutable evidence and normalized event stores.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Model and prompt versions logged

45. **Place rule, ML and LLM services behind explicit interfaces.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Tool actions use scoped identity

46. **Add case-management evidence links and a tool-broker approval gate.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Manual fallback documented

47. **Document identity, logging, fallback and recovery paths.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Manual fallback documented

48. **Validate the architecture against the provided control checklist.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Manual fallback documented

### Run and verify

```
cd labs/lab-08-design-an-ai-enhanced-soc-architecture
python3 solution/main.py
python3 -m pytest -q tests
```

### Acceptance criteria

- Raw evidence is immutable
- Model and prompt versions logged
- Tool actions use scoped identity
- Manual fallback documented

### Troubleshooting

If the script cannot find data, run it from the lab folder. If scores are unexpected, inspect type conversion, null handling, baseline/evaluation split and threshold direction. Do not lower a threshold merely to make a test pass; explain the operational effect.


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

## Lab 9: Build a Grounded Threat-Intelligence Assistant

Mapped outcome: LO3 · Tools: Python 3 · TF-IDF · local documents · Hacklab: Ethical Hacking Lab Simulator

Retrieve from approved threat bulletins and produce cited, abstaining summaries without external APIs.

### Controlled simulator or Kali-style practice

Use Hacklab: Ethical Hacking Lab Simulator (https://alfredang.github.io/ethnicalhacking/) in its Reconnaissance scenario module. Capture simulated reconnaissance facts, convert them into approved local bulletins and test citation and abstention behaviour. Follow tool-exercise.md in the individual lab folder. The files in mock-data/ provide a fully offline assessment-equivalent path.

Safety boundary: use the browser simulation or a trainer-owned isolated Kali target only. Do not scan or test unrelated live systems.

### Detailed step-by-step procedure

49. **Review the approved local threat bulletins and document ACL metadata.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Every claim has document citation

50. **Chunk and index the documents with TF-IDF.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — ACL filtering occurs before retrieval

51. **Retrieve top-k evidence for each analyst query.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Unknown query abstains

52. **Generate an extractive summary that cites document IDs.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Embedded instructions are treated as data

53. **Return 'insufficient evidence' below the similarity threshold.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Embedded instructions are treated as data


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

54. **Test forbidden-document and indirect-instruction cases.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Embedded instructions are treated as data

### Run and verify

```
cd labs/lab-09-build-a-grounded-threat-intelligence-assistant
python3 solution/main.py
python3 -m pytest -q tests
```

### Acceptance criteria

- Every claim has document citation
- ACL filtering occurs before retrieval
- Unknown query abstains
- Embedded instructions are treated as data

### Troubleshooting

If the script cannot find data, run it from the lab folder. If scores are unexpected, inspect type conversion, null handling, baseline/evaluation split and threshold direction. Do not lower a threshold merely to make a test pass; explain the operational effect.

## Lab 10: Design a Guarded SOAR Playbook

Mapped outcome: LO3 · Tools: Python 3 · JSON · FauxBank Cybersecurity Training Simulation

Build a reversible, approval-gated response playbook for suspicious account activity.

### Controlled simulator or Kali-style practice

Use FauxBank Cybersecurity Training Simulation (https://pentest-fauxbank.vercel.app/) in its Simulated scanner and report generator module. Convert selected fictional findings into guarded response cases; keep all actions in dry-run or submit-for-approval mode. Follow tool-exercise.md in the individual lab folder. The files in mock-data/ provide a fully offline assessment-equivalent path.

Safety boundary: use the browser simulation or a trainer-owned isolated Kali target only. Do not scan or test unrelated live systems.

### Detailed step-by-step procedure

55. **Define trigger evidence and entity-resolution preconditions.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Default mode is dry-run


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

56. **Add enrichment and confidence checks.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Target identity is verified

57. **Simulate account containment and calculate blast radius.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Approval recorded

58. **Require approval for identity-impacting actions.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Rollback and post-check exist

59. **Define rollback and post-action verification.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Rollback and post-check exist

60. **Run safe dry-run tests; no real accounts are modified.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Rollback and post-check exist

### Run and verify

```
cd labs/lab-10-design-a-guarded-soar-playbook
python3 solution/main.py
python3 -m pytest -q tests
```

### Acceptance criteria

- Default mode is dry-run
- Target identity is verified
- Approval recorded
- Rollback and post-check exist

### Troubleshooting

If the script cannot find data, run it from the lab folder. If scores are unexpected, inspect type conversion, null handling, baseline/evaluation split and threshold direction. Do not lower a threshold merely to make a test pass; explain the operational effect.


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

## Lab 11: Create a 90-Day AI Security Action Plan

Mapped outcome: LO3 · Tools: Python 3 · CSV · Hacklab: Ethical Hacking Lab Simulator

Sequence architecture work packages, dependencies, acceptance evidence and regulatory review triggers.

### Controlled simulator or Kali-style practice

Use Hacklab: Ethical Hacking Lab Simulator (https://alfredang.github.io/ethnicalhacking/) in its Network scanning scenario module. Use simulated discovery evidence to sequence work packages, dependencies, owners and acceptance gates in a 90-day plan. Follow tool-exercise.md in the individual lab folder. The files in mock-data/ provide a fully offline assessment-equivalent path.

Safety boundary: use the browser simulation or a trainer-owned isolated Kali target only. Do not scan or test unrelated live systems.

### Detailed step-by-step procedure

61. **Import the target architecture work packages.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.

    Check — No dependency cycles

62. **Assign owner, duration, dependency and acceptance evidence.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.

    Check — Every work package has evidence

63. **Detect missing or cyclic dependencies.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.

    Check — Critical path identified

64. **Calculate earliest start and critical path.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.

    Check — Change triggers recorded

65. **Add model/provider change and regulatory-review triggers.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.

    Check — Change triggers recorded


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

66. **Present the 30/60/90-day roadmap for approval.**

Evidence — record the input, configuration or threshold, generated output and decision rationale.
Check — Change triggers recorded

### Run and verify

```
cd labs/lab-11-create-a-90-day-ai-security-action-plan
python3 solution/main.py
python3 -m pytest -q tests
```

### Acceptance criteria

- No dependency cycles
- Every work package has evidence
- Critical path identified
- Change triggers recorded

### Troubleshooting

If the script cannot find data, run it from the lab folder. If scores are unexpected, inspect type conversion, null handling, baseline/evaluation split and threshold direction. Do not lower a threshold merely to make a test pass; explain the operational effect.

## Topic 4: Monitoring, Evaluating and Governing AI Security Solutions

Alignment: LO4. Metrics · drift · adversarial testing · standards · advisory reporting

### Confusion-matrix metrics connect detection to operations

Runtime mechanism: Labels -> Predictions -> TP/FP/FN/TN -> Metrics -> Operating point

| Contract field / rule | Operational meaning |
|---|---|
| precision=TP/(TP+FP) | Accuracy hides class imbalance |
| recall=TP/(TP+FN) | Precision measures alert usefulness |
| FPR=FP/(FP+TN) | Recall measures missed-threat exposure |
| cost = FP*c_fp + FN*c_fn | Cost weights business consequence |

### Failure, control and evidence

| Failure mode | Control / evidence |
|---|---|
| Evaluation set not representative | Frozen test set |
| Labels from same detector | Class prevalence reported |
| Metric without confidence interval | Operating threshold approved |
| Threshold changed without review | Metric tied to workload |

Practice bridge: Lab 12 applies this mechanism; the full procedure follows in this guide and the individual lab folder.


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

## Drift monitoring compares live distributions and outcomes

Runtime mechanism: Reference -> Live window -> Feature drift -> Performance drift -> Response

| Contract field / rule | Operational meaning |
|---|---|
| PSI = Σ(actual-expected) ln(actual/expected) | Input drift is an early warning |
| feature null rate | Performance needs delayed labels |
| score distribution | Seasonality needs matched baselines |
| confirmed precision | Retraining is not the only response |

### Failure, control and evidence

| Failure mode | Control / evidence |
|---|---|
| Alert fatigue from noisy drift | Drift thresholds |
| Silent schema change | Schema-change alert |
| Poisoned feedback | Label quality sample |
| Automatic retraining on attacks | Rollback/recalibrate decision |

Practice bridge: Lab 12 applies this mechanism; the full procedure follows in this guide and the individual lab folder.

## Grounding and citation tests manage hallucination risk

Runtime mechanism: Question -> Retrieve evidence -> Answer -> Cite -> Verify/abstain

| Contract field / rule | Operational meaning |
|---|---|
| claim span | Faithfulness differs from usefulness |
| source document/version | Citation presence is not citation correctness |
| entailment result | Unsupported claims can be blocked |
| abstention reason | High-impact advice requires review |

### Failure, control and evidence

| Failure mode | Control / evidence |
|---|---|
| Fabricated IOC | Claim-level citation test |
| Citation points to irrelevant paragraph | Freshness SLO |
| Stale policy | Known-unanswerable set |
| Answer exceeds source | Analyst sign-off |

Practice bridge: Lab 10 applies this mechanism; the full procedure follows in this guide and the individual lab folder.

## Adversarial testing follows attacker goals and trust boundaries

Runtime mechanism: Threat model -> Test cases -> Execute -> Observe -> Remediate/retest

| Contract field / rule | Operational meaning |
|---|---|
| ATLAS technique | Direct injection targets user input |
| precondition | Indirect injection arrives via retrieved content |
| payload/source | Evasion perturbs model features |
| expected control | Poisoning targets training/feedback |
| evidence | Poisoning targets training/feedback |

### Failure, control and evidence

| Failure mode | Control / evidence |
|---|---|
| Testing production destructively | Isolated test tenant |
| No data exfiltration canary | Canary secret |
| Tool-call side effect | Tool dry-run |


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

| Failure mode | Control / evidence |
|---|---|
| Payloads retained in logs | Retest closes finding |

Practice bridge: Lab 12 applies this mechanism; the full procedure follows in this guide and the individual lab folder.

## Privacy and data-loss controls apply before, during and after inference

Runtime mechanism: Discover -> Classify -> Redact -> Authorise -> Monitor/delete

| Contract field / rule | Operational meaning |
|---|---|
| secret/PII detectors | Prevention starts before the prompt |
| token-level redaction | Outputs can reconstruct inputs |
| purpose + consent | Logs may be more sensitive than prompts |
| retention/deletion receipt | Deletion spans provider and local stores |

### Failure, control and evidence

| Failure mode | Control / evidence |
|---|---|
| Prompt contains API key | Secret canary blocked |
| Model memorisation | Output DLP test |
| Sensitive output | Log access review |
| Debug log leakage | Deletion proof |

Practice bridge: Lab 8 applies this mechanism; the full procedure follows in this guide and the individual lab folder.

## Standards mapping turns principles into evidence

Runtime mechanism: Requirement -> Control -> Implementation -> Evidence -> Finding

| Contract field / rule | Operational meaning |
|---|---|
| framework/function/category | NIST CSF structures cyber outcomes |
| control owner | AI RMF covers AI-specific governance |
| test frequency | ISO-style controls support assurance |
| evidence location | Mapping avoids duplicate evidence |

### Failure, control and evidence

| Failure mode | Control / evidence |
|---|---|
| Checkbox mapping | Sample evidence |
| No test frequency | Independent test |
| Vendor assertion only | Exception record |
| Conflicting control owners | Management review |

Practice bridge: Lab 12 applies this mechanism; the full procedure follows in this guide and the individual lab folder.

## Continuous assurance combines technical tests and operational review

Runtime mechanism: Control test -> Detection test -> Scenario exercise -> Metric review -> Improvement

| Contract field / rule | Operational meaning |
|---|---|
| test id/version | Unit tests catch contract drift |
| expected result | Purple-team tests validate detections |
| observed evidence | Tabletops validate decisions |
| owner/due date | Metrics expose operating degradation |


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

### Failure, control and evidence

| Failure mode | Control / evidence |
|---|---|
| Tests never fail | Failure injection |
| Synthetic scenario unlike production | Independent review |
| Finding closed without evidence | Retest evidence |
| No retest | Backlog trend |

Practice bridge: Lab 12 applies this mechanism; the full procedure follows in this guide and the individual lab folder.

## Advisory reporting separates facts, inference and recommendations

Runtime mechanism: Executive outcome -> Evidence -> Risk -> Options -> Recommendation

| Contract field / rule | Operational meaning |
|---|---|
| finding + source | Facts remain reproducible |
| confidence/limitations | Inference is labelled |
| decision owner | Options expose trade-offs |
| action/measure/date | Recommendation has owner and metric |

### Failure, control and evidence

| Failure mode | Control / evidence |
|---|---|
| AI-generated certainty | Source appendix |
| No limitation statement | Assumption register |
| Recommendation without cost | Management decision |
| No follow-up measure | 30/60/90-day readback |

Practice bridge: Lab 12 applies this mechanism; the full procedure follows in this guide and the individual lab folder.

## Lab 12: Monitor Model Effectiveness and Produce Advice

Mapped outcome: LO4 · Tools: Python 3 · pandas · scikit-learn metrics · Cryptography Toolkit

Measure detection quality, identify drift and control failures, then issue evidence-backed adoption advice.

### Controlled simulator or Kali-style practice

Use Cryptography Toolkit (https://alfredang.github.io/cryptography-toolkit/) in its ECDSA sign and verify module. Sign the fictional advisory report, verify the original, then alter one statement and record the expected verification failure as integrity evidence. Follow tool-exercise.md in the individual lab folder. The files in mock-data/ provide a fully offline assessment-equivalent path.

Safety boundary: use the browser simulation or a trainer-owned isolated Kali target only. Do not scan or test unrelated live systems.


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

### Detailed step-by-step procedure

67. **Build the confusion matrix from labelled outcomes.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Metrics include class prevalence

68. **Calculate precision, recall, F1 and false-positive rate.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Drift is diagnostic, not automatic retraining

69. **Compare feature distributions with the reference window.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Advice separates fact/inference/recommendation

70. **Review prompt-injection, citation and permission tests.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Each action has owner and measure

71. **Map findings to NIST CSF/AI RMF control outcomes.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Each action has owner and measure

72. **Write a recommendation with options, limitations, owner, metric and 30/60/90-day follow-up.**

    Evidence — record the input, configuration or threshold, generated output and decision rationale.
    Check — Each action has owner and measure

### Run and verify

```
cd labs/lab-12-monitor-model-effectiveness-and-produce-advice
python3 solution/main.py
python3 -m pytest -q tests
```

### Acceptance criteria

- Metrics include class prevalence
- Drift is diagnostic, not automatic retraining
- Advice separates fact/inference/recommendation
- Each action has owner and measure


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

### Troubleshooting

If the script cannot find data, run it from the lab folder. If scores are unexpected, inspect type conversion, null handling, baseline/evaluation split and threshold direction. Do not lower a threshold merely to make a test pass; explain the operational effect.

## References

Retrieval date: 28 August 2026.


- ACSC et al. (2026). Opportunities for AI in cyber defence. https://www.cyber.gov.au/business-government/secure-design/artificial-intelligence/opportunities-for-ai-in-cyber-defence
- NIST (2024). Cybersecurity Framework 2.0. https://www.nist.gov/cyberframework
- NIST (2023). AI Risk Management Framework 1.0. https://www.nist.gov/itl/ai-risk-management-framework
- NIST (2025). Adversarial Machine Learning Taxonomy and Terminology. https://csrc.nist.gov/pubs/ai/100/2/e2025/final
- MITRE. ATLAS: Adversarial Threat Landscape for AI Systems. https://atlas.mitre.org/
- OWASP. Top 10 for Large Language Model Applications. https://genai.owasp.org/llm-top-10/
- CISA. Secure by Design. https://www.cisa.gov/securebydesign
- Google Cloud. AI for Security. https://cloud.google.com/security/ai
- IBM. AI cybersecurity solutions. https://www.ibm.com/solutions/ai-cybersecurity
- Red Hat. Four use cases for AI in cyber security. https://www.redhat.com/en/blog/4-use-cases-ai-cyber-security
- Alfred Ang. Cybersecurity Threat Simulation Lab. https://alfredang.github.io/cybersecuritysimulator/
- Alfred Ang. Hacklab: Ethical Hacking Lab Simulator. https://alfredang.github.io/ethnicalhacking/
- Tertiary Infotech Academy. FauxBank Cybersecurity Training Simulation. https://pentest-fauxbank.vercel.app/
- Alfred Ang. Cryptography Toolkit. https://alfredang.github.io/cryptography-toolkit/



© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg